

Instructivo de Despliegue en Producción

Estructura Arquitectural: Next.js (Vercel) + NestJS & PostgreSQL (Render)

Este documento técnico describe paso a paso el procedimiento estructurado para desplegar la **Versión 1.0 de la Plataforma de Cursos**. La arquitectura se basa en un desacoplamiento completo: el frontend se aloja de forma serverless en Vercel, optimizando la entrega mediante CDN, mientras que la API REST (NestJS) y la base de datos relacional (PostgreSQL) se ejecutan de manera administrada en Render.

1. Requisitos Previos Estratégicos

Antes de comenzar con la configuración de la infraestructura en la nube, es fundamental verificar que el repositorio de GitHub cuente con la estructura adecuada:

- **Estructura del Proyecto:** Si utilizas un monorepo (frontend y backend en la misma raíz), asegúrate de conocer los paths de cada subcarpeta (ej. `/apps/frontend` o `/backend`). Si son repositorios independientes, el proceso será aún más directo.
- **Configuración de CORS:** En el backend (NestJS), debes asegurarte de que la inicialización del servidor permita solicitudes desde dominios externos, o configurar el comodín de producción una vez que conozcas la URL de Vercel.
- **Cuentas Activas:** Acceso administrativo a las plataformas [Render.com](#) y [Vercel.com](#), vinculadas de manera preferencial a la misma cuenta de GitHub para facilitar los permisos de lectura.

2. Fase I: Despliegue de la Base de Datos (Render)

Es indispensable inicializar primero la base de datos, ya que el backend requerirá la cadena de conexión activa de inmediato para superar la fase de construcción (build) si tienes validaciones de conexión durante el inicio o migraciones automáticas.

1. Inicia sesión en el panel de control de **Render**.
2. Haz clic en el botón **"New +"** (esquina superior derecha) y selecciona la opción **"PostgreSQL"**.
3. Completa el formulario de aprovisionamiento con los siguientes parámetros técnicos:

Campo	Valor Recomendado / Acción
Name	<code>plataforma-cursos-db-prod</code>
Database	<code>courses_prod_db</code> (Nombre interno)
User	Déjalo vacío para que el sistema genere un usuario seguro por defecto.
Region	Selecciona la más cercana a tu tráfico meta o la misma donde pondrás la API (ej. <code>Oregon (US West)</code> o <code>Ohio (US East)</code>).
Instance Type	Para producción estable, selecciona el plan Starter (evita el plan free ya que se suspende tras 90 días).

4. Haz clic en **"Create Database"** al final del formulario.
5. **Extracción de Credenciales:** Una vez que el estado de la base de datos cambie a `Available`, desplázate hacia abajo hasta la sección **"Connections"**. Copia el valor de **"Internal Database URL"** si tu API estará dentro de Render (es más rápido y seguro) o el **"External Database URL"** para conexiones externas de mantenimiento local.

Nota de Arquitectura: La cadena de conexión tendrá una estructura similar a:

`postgresql://user:password@hostname:port/database_name`. Manténla a resguardo; la inyectaremos en la API en el siguiente paso.

3. Fase II: Despliegue de la API REST (NestJS en Render)

Con la base de datos activa, procedemos a conectar la API construida en NestJS directamente desde el control de versiones de GitHub.

1. En el panel principal de Render, haz clic en **"New +"** y selecciona **"Web Service"**.
2. Selecciona la opción **"Connect a repository"**. Si es la primera vez que lo utilizas, otorga permisos a Render para acceder al repositorio específico de la plataforma de cursos de tu cuenta de GitHub.
3. Configura las propiedades principales del servicio web:
 - **Name:** `plataforma-cursos-api-prod`
 - **Region:** Asegúrate de que sea estrictamente la **"misma región"** elegida para la base de datos PostgreSQL para reducir la latencia de red a menos de 1ms.
 - **Branch:** `main` (o tu rama de producción configurada).
 - **Root Directory:** Si tu código de NestJS no está en la raíz, especifica la carpeta (ej. `backend`). De lo contrario, déjalo vacío.
 - **Runtime:** `Node`
4. **Comandos de Construcción y Ejecución:** Configura con precisión los comandos del ciclo de vida de Node.js:

Build Command:

```
npm install && npm run build
```

Start Command:

```
npm run start:prod
```

Alternativa de optimización para producción: Si tu script de build compila directamente a código nativo de Node dentro de una carpeta `dist/`, un comando de inicio más eficiente y con menor consumo de memoria es `node dist/main.js`.

5. **Instancia y Recursos:** Selecciona el plan comercial adecuado (ej. plan **Starter** o superior para producción, garantizando RAM dedicada y que el contenedor no entre en modo de hibernación por inactividad).

6. **Configuración de Variables de Entorno (Crucial):** Haz clic en la sección **"Advanced"** e introduce las siguientes claves de entorno:

Variable / Key	Valor / Origen del Dato
DATABASE_URL	Pega aquí el Internal Database URL copiado en la Fase I.
NODE_ENV	production
PORT	10000 (O el puerto asignado internamente por NestJS. Render inyecta dinámicamente un puerto, asegúrate de que tu <code>main.ts</code> escuche en <code>process.env.PORT 3000</code>).

7. Haz clic en **"Create Web Service"**. Render comenzará a descargar el repositorio, ejecutar el empaquetado de NestJS y poner la API en línea. Copia la URL pública generada (ej. `https://plataforma-cursos-api-prod.onrender.com`).

Aviso de Seguridad en NestJS: Verifica que en tu archivo `main.ts` el método de escucha esté correctamente adaptado para producción:

```
const port = process.env.PORT || 3000;
await app.listen(port, '0.0.0.0'); // Escuchar en todas las interfaces de red de
Render
```

4. Fase III: Despliegue del Frontend (Next.js en Vercel)

El frontend se desplegará de forma paralela en Vercel, consumiendo la API de Render mediante llamadas seguras del lado del servidor (SSR/ISR) o peticiones del cliente.

1. Accede a tu cuenta de **Vercel** y haz clic en el botón **"Add New..."** -> **"Project"**.
2. Busca el repositorio de GitHub de la plataforma de cursos y haz clic en **"Import"**.
3. En la configuración del proyecto:
 - **Framework Preset:** Vercel detectará automáticamente `Next.js` de forma nativa.
 - **Root Directory:** Si el frontend está alojado en una subcarpeta, haz clic en **"Edit"** y selecciónala (ej. `apps/frontend`).
4. **Build and Output Settings:** Déjalos por defecto. Vercel conoce de forma nativa los comandos optimizados (`next build`).
5. **Environment Variables:** Despliega esta sección e introduce las variables requeridas por tu código para conectarse al backend en producción:

Clave de Entorno	Valor Técnico
NEXT_PUBLIC_API_URL	Pega la URL pública generada por Render en el paso anterior (ej. <code>https://plataforma-cursos-api-prod.onrender.com</code>). Asegúrate de omitir la barra diagonal final (/) si tus servicios la concatenan manualmente.

6. Haz clic en el botón **"Deploy"**. Vercel compilará la aplicación en segundos y generará las funciones Serverless necesarias junto con una URL de producción pública protegida por certificado SSL automático.

5. Fase IV: Ajuste Final de CORS y Verificación

Para garantizar que las peticiones hechas desde el navegador de los alumnos no sean rechazadas por políticas de seguridad del navegador, realiza esta última validación:

- Copia la URL final que te entregó Vercel (ej. `https://plataforma-cursos.vercel.app` o el dominio personalizado de tu cliente).
- Si configuraste los orígenes permitidos de CORS de forma estricta en el archivo `main.ts` de NestJS, actualiza la variable de entorno en Render destinada para los orígenes permitidos (ej. `CORS_ORIGIN=https://plataforma-cursos.vercel.app`) y aplica los cambios (Redeploy).

Monitoreo Continuo: A partir de este momento, cualquier cambio que subas a la rama `main` de tu repositorio en GitHub disparará automáticamente un nuevo despliegue tanto en Render como en Vercel de forma transparente y sin caídas de servicio.